

Performance comparative of OpenCV Template Matching method on Jetson TX2 and Jetson Nano developer kits

Artiom Basulto-Lantsova
Dept. of Electronic Engineering
TecNM en Celaya
Celaya, Mexico
m1803108@itcelaya.edu.mx

Jose A. Padilla-Medina
Dept. of Electronic Engineering
TecNM en Celaya
Celaya, Mexico
alfredo.padilla@itcelaya.edu.mx

Francisco J. Perez-Pinal
Dept. of Electronic Engineering
TecNM en Celaya
Celaya, Mexico
francisco.perez@itcelaya.edu.mx

Alejandro I. Barranco-Gutierrez
Dept. of Electronic Engineering
TecNM en Celaya
Celaya, Mexico
israel.barranco@itcelaya.edu.mx

Abstract—Template Matching is a widely used method for object detection in digital images, it requires great processing power since it is an exhaustive method that compares the intensity levels of a source image pixel-to-pixel with a template image that contains the object to identify. Nowadays there are dedicated embedded systems that provide high processing capabilities, such as the NVIDIA Jetson family. This work presents the experimentation and the performance comparison between the Jetson Nano and Jetson TX2 development kits, when implementing the Template Matching method, in order to get an evaluation criterion to select one of them in image processing projects. It was carried out to six images with different sizes and two variants in terms of the size of the template image. The processing times for the sequential implementation using the CPUs and the parallel implementation with the GPUs were obtained quantitatively. It was observed that the processing times using the parallel versions on average doubled those of the sequential versions and that the Jetson TX2 exceeded the Jetson Nano in execution speeds.

Index Terms—Computer Vision; Template Matching; NVIDIA Jetson; Performance; OpenCV; C++; Sequential processing; Parallel processing

I. INTRODUCTION

In the area of image processing several methods exist used to perform object detection, such as color-based segmentation [1], key point and descriptors-based methods like corners and edges [2], based on Haar-like features [3] and the so-called Template Matching (TM) [4]. Among them, TM is one of the most used, because it is the ideal method for situations in which the other methods present difficulties, such as working with grayscale or noisy images and search for highly symmetrical objects, being also simpler [5]. Despite presenting limitations when there are rotations or changes of scale in the objects, many works have addressed these problems, proposing methods that give solutions to them [6]–[8]. This method basically compares

a window called template image with a segment of the same size from a source image, in order to find the object, present in the template. Performing the pixel-to-pixel comparison between the template and sliding window from the source image implies a high computation time, which can be optimized by parallelizing the processing, thus accelerating it.

Currently, the use of embedded systems such as those of the NVIDIA Jetson family [9] has intensified because they are high performance and low power consumption systems, these provide a large processing capacity and allow the execution of functions in parallel because they are platforms with Graphics Processing Units (GPU), in addition to their portability. Another advantage is that Jetson platforms are compatible with the Jet-Pack Software Development Kit (SDK), it has libraries for deep learning, computer vision, accelerated computing, among others [10]. This has caused Jetsons to be used in various applications, including autonomous vehicle vision systems [11]–[13], quality control systems [14], medical systems [15], [16], and others [17]–[19], using TM in most cases.

Jetsons have the Linux operating system, allowing the use of many useful libraries for image processing. One of the most used is Open Source Computer Vision Library (OpenCV), which is a multi platform computer vision library. It was created with the objective of providing a common infrastructure for computer vision applications [20]. Many authors [21]–[26] make use of this library due to the large number of functions that are included in it that facilitate image processing. In this work, it is used the OpenCV TemplateMatching function.

It is common to find various performance comparisons when implementing these methods sequentially using the Central Processing Units (CPU) and in parallel with GPUs. In [5] they implement a fully parallelizable method of multi-scale TM and they compare it with a sequential implementation, resulting in a 16 times reduction in processing time when using the GPU accelerated version. The same authors, in [27] present two meth-

ods (multi window and partial summary) to parallelize the TM and to compare them with a sequential solution, obtaining an improvement of up to 10 times less processing time obtained with the sequential variant. On the other hand, in [25] they carried out a performance comparison between various functions used for image processing operations that are part of OpenCV for CPU and GPU, resulting in an improvement of the performance of the functions implemented in the GPU, highlighting that greater effects are found by increasing the size of the image.

This work presents a performance comparison between the Jetson TX2 [28] and Jetson Nano [29] developer kits in terms of the execution times of the TemplateMatching functions for CPU and GPU with OpenCV. The same is done with images of different sizes, to measure as soon as it affects an increase in the size of the image in the execution times and if there is a marked difference in performance between both developer kits. Two tests were performed, the first with template images of equal size for all source images and the second with template images of varying size, the size relationship between the source image and the template image being equal in all cases, with the aim of making a fair comparison between the two platforms.

II. TEMPLATE MATCHING

In general TM it is a technique that consists of finding areas of an image that matches a template image. For this there are two key elements [30]:

- The source image (I): The image where you expect to find the object of the template image.
- Template image (T): The image of the patch to be compared, in order to identify the largest area of coincidence.

To find the area of coincidence, the template image is compared with the source image in the form of a sliding window as shown in Fig. 1, moving the patch one pixel to pixel from left to right and from top to bottom.

A metric is calculated for each location to represent how similar is the patch to that particular area of the source image. It is stored in a result array, in this the highest or lowest value is saved depending on the method.

OpenCV has six methods to calculate the similarity. Which are presented below [30]:

- 1) TM_SQDIFF (Squared difference).
- 2) TM_SQDIFF_NORMED (Normalized squared difference).
- 3) TM_CCORR (Cross correlation).
- 4) TM_CCORR_NORMED (Normalized cross correlation).
- 5) TM_CCOEFF (Cosine coefficient).
- 6) TM_CCOEFF_NORMED (Normalized cosine coefficient).

III. MATERIALS AND METHODS

To implement TM method, six images of different sizes were used. The main image was obtained with a Go Pro 7 camera in 4K resolution and the rest were obtained by processing it to reduce its size. Template images were extracted from each of the images independently.

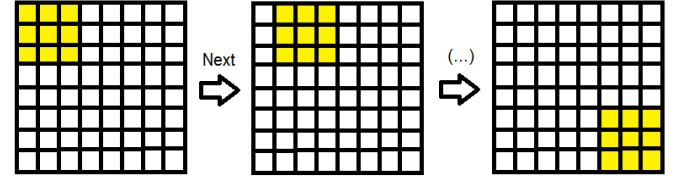


Fig. 1. Sliding window. The yellow region being the window that represents the template image and the rest is the source image.

Two cases were analysed regarding the size of the template images to use.

- 1) Template images of equal size for all cases.
- 2) Template images of variable size, increasing the size proportionally according to the increase in resolution.

For the first case, 50x50 pixels constant template images were taken. For the second case, different sizes of template were used. The 4K image and a template image of 300x300 pixels were taken as the basis, the calculation to obtain the dimensions of the other template images are shown in (1) and (2), being w_{tx} , h_{tx} , $4w_{ix}$, $4h_{ix}$ the width and height of the template image and source image for which the calculation is made, and w_{i4k} , h_{i4k} , w_{i4k} , h_{i4k} the dimensions of the template image and 4K source image. Table I shows the relationships between the source images and the template images and their dimensions.

$$w_{tx} = w_{i4k} / ((w_{i4k} / w_{ix})) \quad (1)$$

$$h_{tx} = h_{i4k} / ((h_{i4k} / h_{ix})) \quad (2)$$

The six template methods were applied to all images to calculate the similarities. Both kits were configured to work at their maximum performance, making use of the jetson_clocks.sh script present in both, which maximizes the performance since it sets the CPU and GPU clocks to the maximum frequency, Table II shows the characteristics of both kits. Their programming was done with the help of the C++ language.

TABLE I
SOURCE IMAGE AND TEMPLATE IMAGE SIZE

Image resolutions				
<i>I</i>	<i>Size(pixels)</i>	<i>T</i>	<i>C_1(pixels)</i>	<i>C_2(pixels)</i>
img_4k.png	3840x2160	4k.png	50x50	300x300
img_2k.png	2560x1440	2k.png	50x50	200x200
img_fhd.png	1920x1080	fhd.png	50x50	150x150
img_hd.png	1280x720	hd.png	50x50	100x100
img_sd.png	720x480	sd.png	50x50	57x67
img_qvga.png	320x240	qvga.png	50x50	25x34

IV. RESULTS

The processing times obtained by carrying out the different implementations for each image in both development kits are shown in Fig. 2-7. Each figure includes the processing times when using the CPU and GPU in the two proposed cases.

TABLE II
FEATURES OF JETSON DEVELOPMENT KITS

Hardware features	Jetson Nano	Jetson TX2
CPU	4-core ARM A57 @ 1.43GHz	4-core ARM Cortex-A57 @ 2GHz, 2-core Denver2 @ 2GHz
GPU	128-core Maxwell @ 921MHz	256-core Pascal @ 1.3GHz
Memory	4GB LPDDR4, 25.6 GB/s	8GB 128-bit LPDDR4, 59.7 GB/s
Power	5W or 10W	7.5W or 15W

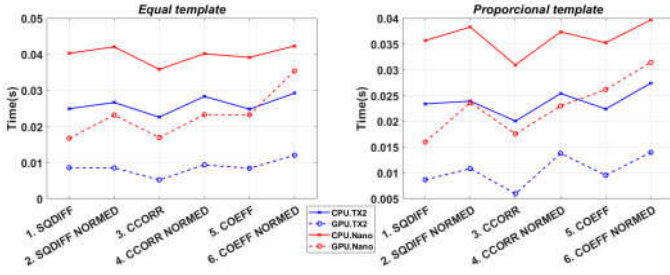


Fig. 2. Processing times in QVGA source image.

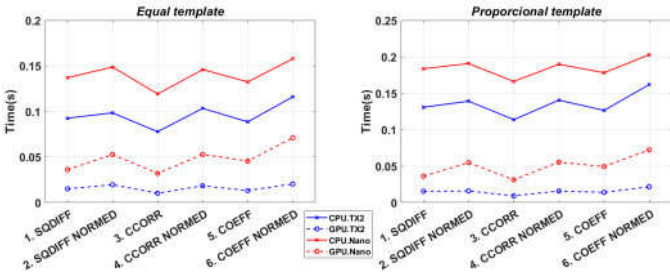


Fig. 3. Processing times in SD source image.

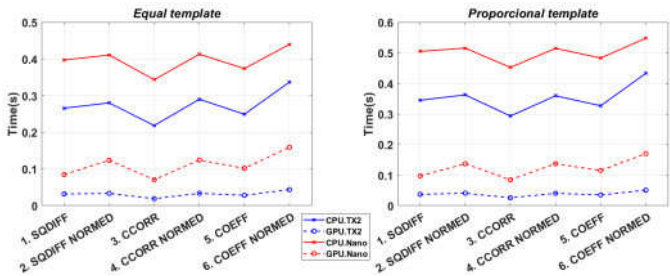


Fig. 4. Processing times in HD source image.

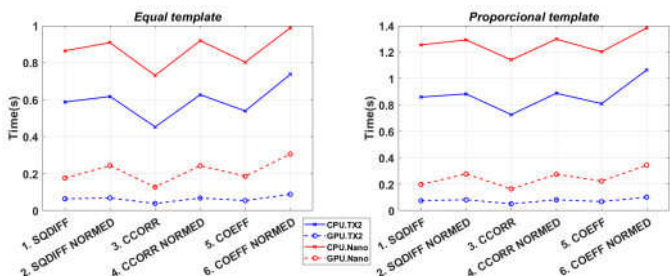


Fig. 5. Processing times in FHD source image.

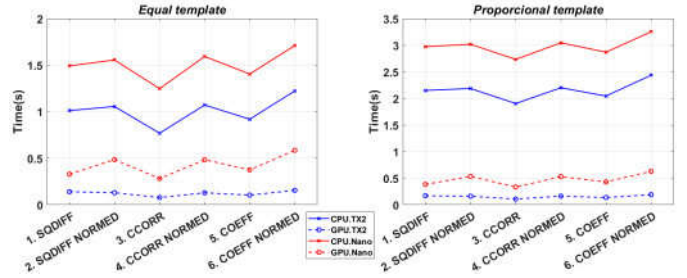


Fig. 6. Processing times in 2K source image.

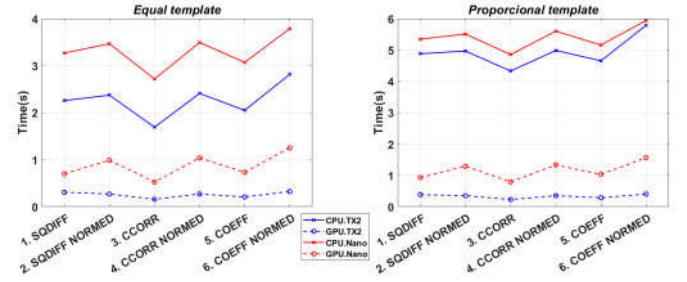


Fig. 7. Processing times in 4K source image.

V. DISCUSSION

A. Equal template images

Using template images of equal resolution for the six source images, it can be seen how in each case the variant implemented with GPU is faster than the implemented with CPU. Noting an increase in the relationship between these execution speeds for higher resolution images, the execution time being approximately double when comparing the processing performed to an image and the one with the highest resolution.

The relationship between the processing time with CPU and GPU presents a growth until reaching the HD image, then with the higher resolution images this relationship remains almost constant for each of the six methods.

As for the execution times when using the Jetson Nano and the Jetson TX2, the latter performs faster processing being between 1.3 and 1.6 times faster with the CPU and between 1.9 and 3.8 times faster with the GPU as shown in Fig. 8 and Fig. 9.

B. Variable dimension template images

In the case of using different sizes of template images according to the resolution of the source image, as in the previous case, the variants implemented with GPU have less processing time than those implemented with CPU for all cases. Likewise, the ratio between these execution speeds for higher resolution images is increased and the execution time is approximately double when the processing is compared on an image and the one with the highest resolution.

The relationship between the processing time with CPU and GPU presents a gradual growth when processed with the Jetson TX2, while with the Jetson Nano there are very small variations from the SD image.

When comparing the execution times in the Jetson Nano and the Jetson TX2, the latter performs faster processing being between 1.0 and 1.6 times faster with the CPU and between 1.6 and 3.8 times faster with the GPU as seen in Fig. 10 and Fig. 11.

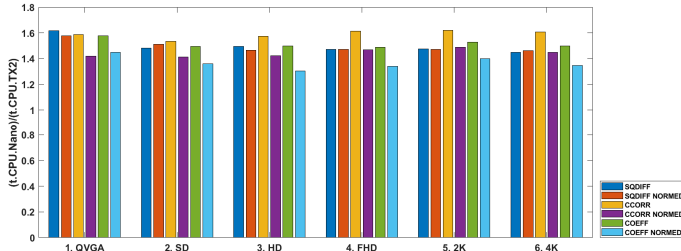


Fig. 8. Performance ratio with CPU (Case 1).

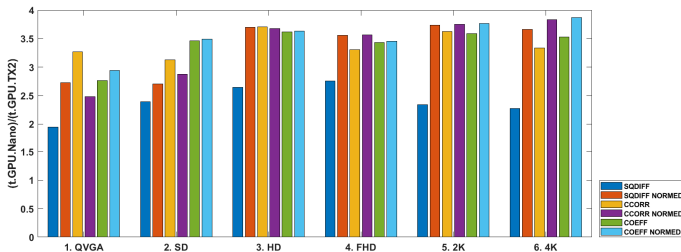


Fig. 9. Performance ratio with GPU (Case 1)

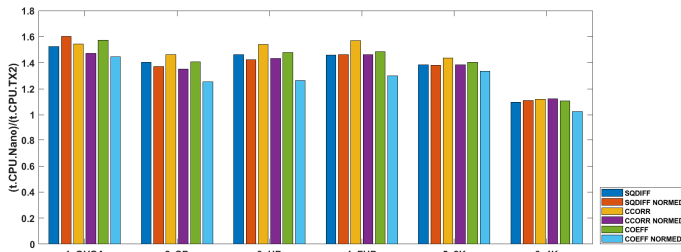


Fig. 10. Performance ratio with CPU (Case 2).

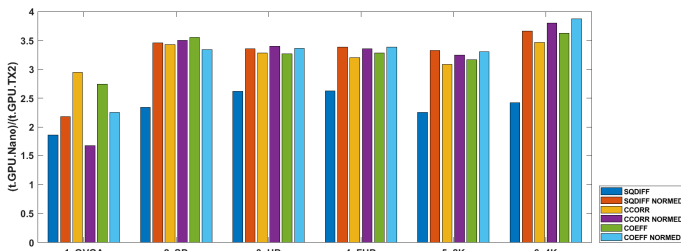


Fig. 11. Performance ratio with GPU (Case 2).

VI. CONCLUSIONS

When comparing the performance of the different TM methods in the Jetson Nano and Jetson TX2 developer kits, it could be seen that, on average, the Jetson TX2 is three times faster than the Jetson Nano for OpenCV correlation processing when GPUs are used. This is an expected result due to their hardware capabilities. However, in both devices the processing times are low enough to make both highly recommended when performing template image processing.

It was also experimented with the CPUs of both kits and although they are slower than GPUs, they also process modestly fast. Regarding the use of a template image of constant or variable size, in the tests carried out there was not a considerable variation in the processing times when one or the other is used. This work is expected to be useful when it is required to decide which kit to use in template image processing projects.

REFERENCES

- [1] L. Barba-Guaman, C. Calderon-Cordova, and P. A. Quezada-Sarmiento, "Detection of moving objects through color thresholding," *Iberian Conference on Information Systems and Technologies, CISTI*, 2017.
- [2] S. Buoncompagni, D. Maio, D. Maltoni, and S. Papi, "Saliency-based keypoint selection for fast object detection and matching," *Pattern Recognition Letters*, vol. 62, pp. 32–40, 2015. [Online]. Available: <http://dx.doi.org/10.1016/j.patrec.2015.04.019>
- [3] S. Guennouni, A. Ahaitouf, and A. Mansouri, "A comparative study of multiple object detection using haar-like feature selection and local binary patterns in several platforms," *Modelling and Simulation in Engineering*, vol. 2015, 2015.
- [4] R. Brunelli, *Template matching techniques in computer vision: theory and practice*. John Wiley & Sons, 2009.
- [5] G. Kertész, S. Szénási, and Z. Vámosy, "Performance Measurement of a General Multi-Scale Template Matching Method," *IEEE 19th International Conference on Intelligent Engineering Systems*, pp. 153–157, 2015.
- [6] B. Yan, L. Xiao, H. Zhang, D. Xu, L. Ruan, Z. Wang, and Y. Zhang, "An adaptive template matching-based single object tracking algorithm with parallel acceleration," *Journal of Visual Communication and Image Representation*, vol. 64, p. 102603, 2019. [Online]. Available: <https://doi.org/10.1016/j.jvcir.2019.102603>
- [7] S.-h. Kim, S.-h. Song, J.-h. Kim, Z. Yuan, and J.-d. Cho, "Fast Rotation-Invariant Template Matching with Candidate Reduction Using CUDA," no. 1, pp. 1–2, 2015.
- [8] F. Mendez, M. Ferguson, and S. Tancock, "Real-Time User Selected Dynamic Template Tracking For UAV," no. section VI, 2017.
- [9] NVIDIA, "NVIDIA Jetson," 2019. [Online]. Available: <https://developer.nvidia.com/embedded/develop/hardware> Accessed 21-Feb-2019
- [10] —, "Autonomous machines," 2019. [Online]. Available: <https://www.nvidia.com/es-la/autonomous-machines/embedded-systems/> Accessed on 23-Sep-2019.
- [11] N. Cooper, E. Lindsey, R. Chapman, and S. Biaz, "GPU Based Monocular Vision for Obstacle Detection," *CSSE17-08*, pp. 1–14, 2017.
- [12] A. Shustanov and P. Yakimov, "A method for traffic sign recognition with CNN using GPU," *ICETE 2017 - Proceedings of the 14th International Joint Conference on e-Business and Telecommunications*, vol. 5, no. Icete, pp. 42–47, 2017.
- [13] N. Ottensmeyer, M. Yang, S. Rust, E. Park, J. H. Anderson, F. D. Smith, A. Berg, and S. Wang, "An Evaluation of the NVIDIA TX1 for Supporting Real-time Computer-Vision Workloads," *2017 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pp. 355–365, 2017.
- [14] A. Elkaseer, M. Salama, S. Scholz, A. Elkaseer, H. Ali, M. Salama, and S. Scholz, "Approaches to a Practical Implementation of Industry 4.0," *The Eleventh International Conference on Advances in Computer-Human Interactions*, no. March, pp. 141–146, 2018.
- [15] W. J. Chang, L. B. Chen, C. H. Hsu, C. P. Lin, and T. C. Yang, "A Deep Learning-Based Intelligent Medicine Recognition System for Chronic Patients," *IEEE Access*, vol. 7, no. c, pp. 44 441–44 458, 2019.

- [16] J. Sancho Aragón, "Energy and Performance Modeling of NVIDIA Jetson TX1 Embedded GPU in Hyperspectral Image Classification Tasks for Cancer detection Using Machine Learning," Ph.D. dissertation, Escuela Técnica Superior de Ingeniería y Sistemas de Telecomunicación, 2018.
- [17] J. Tang, Y. Ren, and S. Liu, "Real-Time Robot Localization, Vision, and Speech Recognition on Nvidia Jetson TX1," 2017. [Online]. Available: <http://arxiv.org/abs/1705.10945>
- [18] Y. Wu, L. Gao, B. Zhang, B. Yang, and Z. Chen, "Embedded GPU implementation of anomaly detection for hyperspectral images," *High-Performance Computing in Remote Sensing V*, vol. 9646, p. 964608, 2015.
- [19] H. Xing, S. Guo, L. Shi, X. Hou, Y. Liu, Y. Hu, D. Xia, and Z. Li, "Quadrotor Vision-based Localization for Amphibious Robots in Amphibious Area," *2019 IEEE International Conference on Mechatronics and Automation (ICMA)*, pp. 2469–2474, 2019.
- [20] OpenCV, "OpenCV," 2019. [Online]. Available: <https://opencv.org/about/> Accessed on 19-Jun-2019
- [21] N. Boyko, O. Basystiuk, and N. Shakhovska, "Performance Evaluation and Comparison of Software for Face Recognition, Based on Dlib and OpenCV Library," *Proceedings of the 2018 IEEE 2nd International Conference on Data Stream Mining and Processing, DSMP 2018*, pp. 478–482, 2018.
- [22] T. Epanchintsev and A. Sozykin, "Processing large amounts of images on hadoop with OpenCV," *CEUR Workshop Proceedings*, vol. 1513, pp. 137–143, 2015.
- [23] Y.-J. Ha and H.-D. Kang, "Evaluation of feature based image stitching algorithm using OpenCV," pp. 224–229, 2017.
- [24] S. Saxena, S. Sharma, and N. Sharma, "Parallel Image Processing Techniques, Benefits and Limitations," *Research Journal of Applied Sciences, Engineering and Technology*, vol. 12, no. 2, pp. 223–238, 2016.
- [25] B. Hang and Ö. Eyecioğlu, "Performance Comparison Between OpenCV Built in CPU and GPU Functions on Image Processing Operations," *International Journal of Engineering Science and Application*, vol. 1, no. 2, pp. 34–41, 2017.
- [26] C. Zhu, "Indoor positioning and tracking based on OpenCV," Ph.D. dissertation, California State University, Northridge, 2017.
- [27] G. Kertesz, S. Szenasi, and Z. Vamossy, "Parallelization methods of the template matching method on graphics accelerators," *CINTI 2015 - 16th IEEE International Symposium on Computational Intelligence and Informatics, Proceedings*, pp. 161–164, 2015.
- [28] NVIDIA, "Harness AI at the Edge with the Jetson TX2 Developer Kit," 2019. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-tx2-developer-kit>
- [29] —, "Jetson Nano Developer Kit," 2019. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-nano-developer-kit>
- [30] OpenCV, "OpenCV: Template Matching," 2019. [Online]. Available: https://docs.opencv.org/3.4.6/de/da9/tutorial_template_matching.html Accessed on 19-Oct-2019.